



Hyperplane decision trees as piecewise linear surrogate models for chemical process design

Ethan M. Sunshine^a, Carolina Colombo Tedesco^a, Sneha A. Akhade^b, Matthew J. McNenly^b, John R. Kitchin^a, Carl D. Laird^{a,*,*}

^a Chemical Engineering, Carnegie Mellon University, United States of America

^b Lawrence Livermore National Laboratory, United States of America

ARTICLE INFO

Keywords:

Decision trees
Mathematical programming
Mixed-integer linear programming
Machine Learning
Regression

ABSTRACT

Recent trends in chemical engineering research point towards an increasing reliance on data-driven modeling approaches. Neural networks, for instance, have proven to be accurate when data is plentiful and high-dimensional, but in many cases, they require computationally-intensive training procedures. In this work, we describe hyperplane decision trees (HT) as a highly expressive and low-compute machine learning model architecture. These models are locally linear and have linear decision boundaries, resulting in a piecewise linear model of the data. This property allows them to be converted into mixed-integer linear constraints which can be globally optimized. Our open-source PyTorch implementation of this method is a fast, flexible, and accessible way to build accurate piecewise linear models of data.

1. Introduction

It is well-known that piecewise linear approximations are useful in formulating optimization problems in chemical engineering (Cozad et al., 2014; Pedrozo et al., 2020; Grimstad and Andersson, 2019). These approximations, or surrogates, can be used to convert a nonlinear or mixed-integer nonlinear constraint into a mixed-integer linear one. However, there is not a universal method to construct such surrogates. In many cases, one must rely on a data-driven technique such as regression. However, a “piecewise linear regression” is not a trivial problem. Even for a simple problem with one input and one output variable, it is not obvious how to algorithmically determine, from data alone, appropriate endpoints for each linear segment. The problem becomes very challenging as the dimensionality of the input space increases.

It has been shown that this problem can be solved to a globally optimal solution using a mixed-integer programming (MIP) formulation (Teichrib and Darup, 2024). However, this formulation may become computationally intractable for regressions of many data points due to the introduction of many binary variables (the number of data points times the number of possible polytopes). Often, it is necessary to use large numbers of data points to adequately sample a high-dimensional input space. Thus, it is desirable to find piecewise linear regression approaches that scale well to high dimensionality and number of data points, even if these approaches do not necessarily result in the best possible surrogate.

Linear model decision tree (LMDT)s are a regression technique where a domain of data is divided into subdomains with a decision tree, and a linear regression is performed on the data on each subdomain. This enables extremely fast predictions on new data points; simply determine the appropriate subdomain for the new point using the decision tree and make a prediction from the corresponding linear regression. This technique has the flexibility to fit a wide range of nonlinear multivariate functions while retaining simplicity, explainability, and computational efficiency (Raymaekers et al., 2024). The simplicity of the functional form allows the regressions to serve as surrogate constraints in mixed-integer linear optimization problems (Ammari et al., 2023).

LMDTs are built recursively. Beginning with a full domain of data, the training algorithm first considers a finite list of possible binary linear divisions. This is typically implemented as a finite list of quantiles for each input variable. Next, linear regressions are performed on each possible subdomain, and some regression error metric is computed. Then, the possible split with the least error is selected. The resulting subdomains must then be divided with the same procedure, recursively, until a stopping criterion is met. For example, the procedure may stop if there is not enough data in the subdomain to warrant further splitting.

Most existing tree-based surrogate modeling tools are limited to subdomains with rectangular shapes, defined by upper and lower bounds in the input variables. Considering other shapes of subdomains will

* Corresponding author.

E-mail address: claird@andrew.cmu.edu (C.D. Laird).

Table 1
Comparison of methods for building a data-driven piecewise linear surrogate.

Method	Training	Continuity	Subdomains
HT (This work)	Greedy	Discontinuous	Convex
LMDT	Greedy	Discontinuous	Rectangular
ReLU NN	Local optimizer	Continuous	Convex
MIP Formulation (Teichrib and Darup, 2024)	Global optimizer	Discontinuous	Convex
NNRT (Bertsimas et al., 2021)	Local optimizer	Discontinuous	Convex

make an LMDT more expressive, potentially enabling it to fit data more accurately without increasing the number of subdomains. Increasing the size of the tree can be problematic as larger trees are less computationally efficient and more prone to overfitting the data. Small trees are also desirable because the highest-level splits often offer some physical interpretations. Most of all, each subdomain corresponds to a binary variable when translated to a mixed-integer linear constraint for optimization. Increasing the number of binary variables can dramatically increase the computational resources required to solve an optimization problem.

There exist several approaches to building decision trees with hyperplane splits, also known as “oblique decision trees” (Wickramarachchi et al., 2015; Lee and Jaakkola, 2020; Bollwein, 2024; Ganaie et al., 2020; Shah and Sastry, 1999; Setiono and Liu, 1999; Rodriguez et al., 2006; Juez-Gil et al., 2021). In general, most of these methods were developed for decision tree classifiers, or in some cases, decision tree regressors with constant response at each leaf. Since we intend to use our regressed trees as piecewise linear surrogates, we seek an algorithm that is more expressive than these methods by using a linear model at each leaf. Furthermore, the diversity of methods demonstrates that the identification of relevant linear combinations, or hyperplanes, to consider for splitting is a difficult problem. In some cases, non-constant leaves are used for regression, such as in near-optimal nonlinear regression trees (NNRT) (Bertsimas et al., 2021). This requires training by gradient descent, which can sometimes be costly or unreliable. In order to take advantage of the existing and efficient LMDT training algorithms, we must identify a limited number of hyperplanes to sample at the beginning of the training procedure.

In this work, we extend the scope of LMDT subdomains from hyperrectangles to all polytopes. This is achieved by a linear transformation of the input variables prior to regressing the LMDT. This step can be thought of as a feature engineering step, or increasing the dimensionality of the data domain. This increases the cost of building the tree as there are now combinatorially more variables for which bounds must be considered. This also means more linear regressions must be done to evaluate all possible splits, resulting in a significant slowdown. This motivates our reimplementations of the LMDT algorithm in PyTorch, offering better parallelization compared to existing methods. This new regression method, the hyperplane decision tree (HT), is implemented in a new open-source Python package.

The proposed approach has trade-offs compared to the existing methods discussed above. One key difference is training time complexity with respect to the number of data points. The MIP Formulation is hard to scale due to the introduction of integer variables for segment assignment for each data point. The data point clustering introduced in Teichrib and Darup (2024) helps with scaling but the resultant regression is no longer globally optimal. These integer variables mean the training is harder to scale to large datasets compared to ReLU neural networks or NNRTs (Bertsimas et al., 2021), which are both trained with gradient descent algorithms. In contrast, tree-based greedy algorithms generally experience more favorable scaling. A summary of some other important trade-offs is shown in Table 1.

2. Methods

As mentioned, our implementation can be divided into two parts: the linear feature engineering, and the recursive building of the tree. We will summarize each algorithm in this section.

2.1. Linear feature engineering

Consider some data matrix $\mathbf{X}$, with I rows (data points) and J columns (dimensions or features). We wish to find some matrix $\mathbf{A}$ such that the transformed data matrix, $\mathbf{X}' = \mathbf{XA}^\top$, contains a finite but representative sampling of all possible linear transformations of the data. $\mathbf{A}$ will have K rows and J columns, with $K \geq J$, so that $\mathbf{X}'$ will have I rows corresponding to the original data points and K columns corresponding to the transformed (new) features. This section will describe our procedure for obtaining this transformation and is summarized as a flowchart in Fig. 1.

The transformation can also be thought of as a “lifting” of the data into a higher dimensional space. In fact, liftings have been proposed in robust optimization (Rahal et al., 2022; Han and Nohadani, 2024) and stochastic optimization (Georghiou et al., 2015) to find nonlinear and piecewise linear policies. To the best of our knowledge, liftings proposed for these applications are nonlinear transformations of variables into spaces where linear policies can be found. In contrast, our approach uses a linear transformation. This is because the LMDT training method we adapt is only capable of univariate splits, so any potential linear split must be specified explicitly at the beginning of the regression process.

The method of finding representative linear transformations is inspired by Miller index notation (Ashcroft and Mermin, 1976). Two positive integer hyperparameters must be chosen: W , a maximum weight in the linear transformations, and N , the maximum number of terms in each linear transformation. This defines a weight matrix $\mathbf{C}$, with K rows and J columns, where $|C_{kj}| \leq W$ and $1 \leq NZ_k \leq N$, where NZ_k is the number of nonzero terms in each row. This matrix is generated exhaustively by the following procedure: First, the algorithm enumerates all possible size- N combinations with replacement of integers from $-W$ to W , inclusive. Next, all possible permutations of the J columns are enumerated. Each weight combination is applied to each column permutation (outer product) to exhaustively obtain every valid row of $\mathbf{C}$ as previously defined. Some normalization and de-duplication is performed at several intermediate points in this procedure for computational efficiency. Examples of $\mathbf{C}$ matrices are shown in Table 2.

It is important to ensure that the hyperplane weights represent a varied distribution of hyperplane angles in the domain despite the input features potentially having very different magnitudes and ranges. Therefore, the software will also compute the range of each feature $r_j = \max_i(\mathbf{X}_{ij}) - \min_i(\mathbf{X}_{ij})$, which is used to normalize the weights in each column of $\mathbf{C}$. This constructs the final, normalized weight matrix $\mathbf{A}$: $\mathbf{A}_{kj} = \mathbf{C}_{kj}/r_j$. This matrix is computed only once, before the tree is fitted to the data. Any time data is passed through the tree (i.e. for regression or prediction), it is first passed through this linear transformation: $\mathbf{X}' = \mathbf{XA}^\top$.

2.2. Recursive tree building

When the tree is regressed to the data, it is built up by recursively dividing the domain into smaller and smaller subdomains or leaves. This is done by choosing, from a finite list of potential linear splits, the split which leads to the most accurate linear regressions. The regression method is summarized in flowchart form in Fig. 2. This procedure has a few important parameters: S , the minimum number of data points in a leaf; B , the number of quantile splits to consider for each feature, and a loss criterion, the function that is used to evaluate the inaccuracy of the linear regressions.

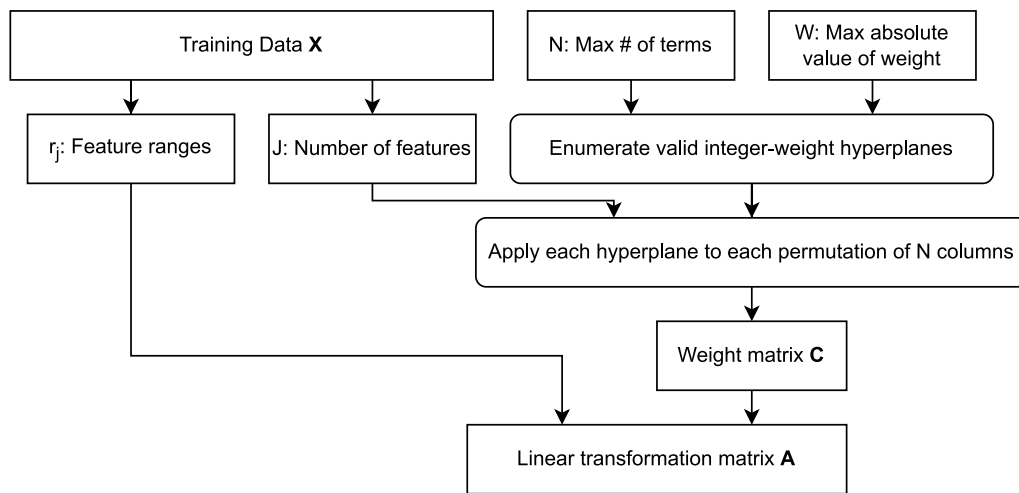


Fig. 1. A flowchart summarizing the information used to produce the weight matrix C and linear transformation matrix A .

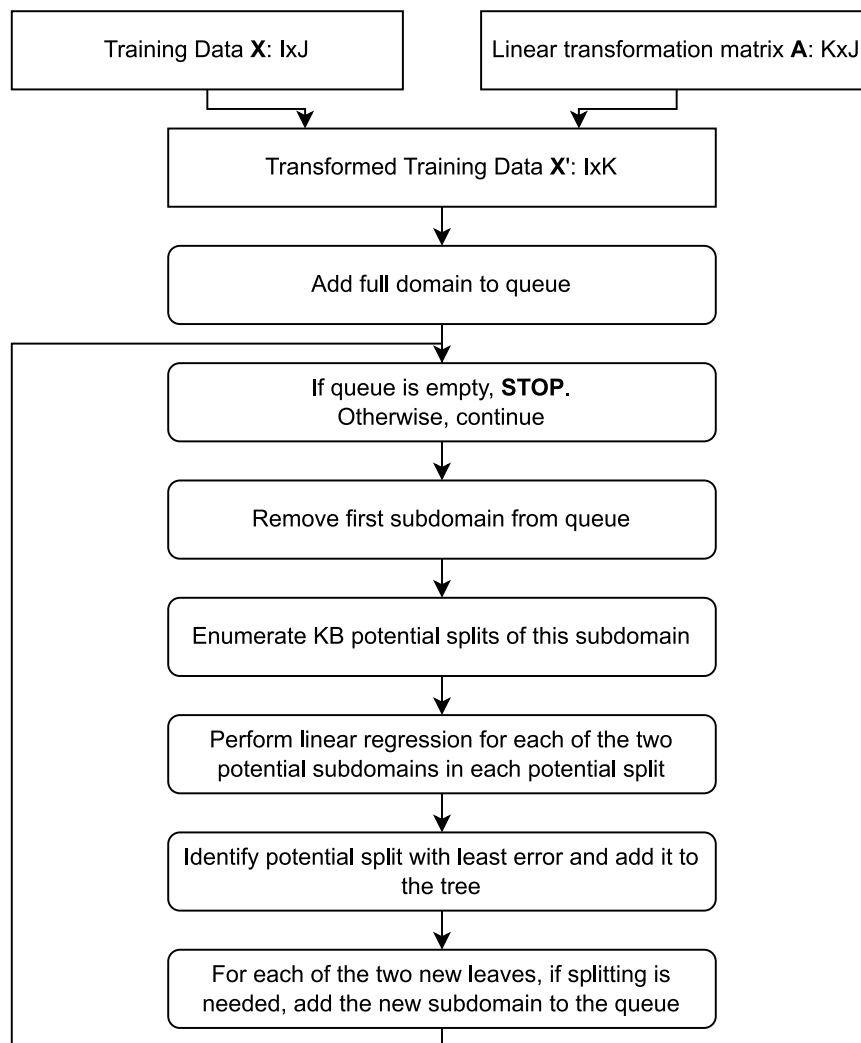


Fig. 2. A flowchart summarizing the recursive tree-building regression method. Note that the last step may result in the addition of zero, one, or two subdomains to the queue, but in all cases, the algorithm proceeds to the next entry in the queue.

Table 2

Example weight matrices generated by this scheme prior to normalization by feature ranges. The first row represents a linear model decision tree with no hyperplanes. The resulting matrix is the identity. In the second row, the $[1, \pm 1, 0]$ hyperplanes are introduced. The third row introduces $[1, \pm 1, \pm 1]$ hyperplanes. Finally, the fourth row demonstrates the effect of increasing W to add the $[1, \pm 2]$ hyperplanes. $J = 3$, $W = 2$, $N = 2$ would result in a C matrix with 21 rows.

J (number of features)	W (max weight)	N (max number of terms)	C (weight matrix)
3	1	1	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
3	1	2	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & -1 & 0 \\ 1 & 0 & -1 \\ 0 & 1 & -1 \end{bmatrix}$
3	1	3	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & -1 & 0 \\ 1 & 0 & -1 \\ 0 & 1 & -1 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \\ 1 & -1 & -1 \end{bmatrix}$
2	2	2	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \\ 1 & 2 \\ 1 & -2 \\ 2 & 1 \\ 2 & -1 \end{bmatrix}$

The first step in the recursive loop is to choose potential binary splits for the current domain. This is done by defining B thresholds for each feature which divide the data into $B + 1$ sets of equal size. Then, each of the B thresholds is used to divide the data into two potential subdomains. Since there are K features in $\mathbf{X}'$, there are KB potential splits defined in this step. A visualization of these splits is shown in Fig. 3.

The transformed data matrix $\mathbf{X}'$ with dimensions $I \times K$ is expanded into a tensor of dimensions $B \times K \times I \times J$. Masks are determined for each of the potential splits and applied according to the appropriate indexing on B and K . Next, a small number ϵ is added to the IJ diagonal to perform a ridge regression. This avoids linear degeneracy issues that can arise when data is generated on a grid (i.e. the x_0 feature will be degenerate with the bias feature if all data points in the subdomain have the same x_0 , which often happens for deep trees of grid-generated data). Putting all of the potential splits into one tensor enables the rapid performance of the $2 \times B \times K$ necessary linear regressions by vectorizing the process into just two calls of `torch.linalg.lstsq` from PyTorch. This function returns a tensor of parameters θ of dimension $B \times K \times J$, representing the linear regression weights for each potential subdomain (only the original J features are used as linear regression features to avoid linear degeneracy). This function is run twice to obtain θ_{above} and θ_{below} .

θ_{above} and θ_{below} are used to make predictions on every data point in their respective subdomains. Again, this process is vectorized as a PyTorch tensor product. The chosen loss criterion function is evaluated, comparing the predictions to the known data. Mean absolute error is

the default loss metric, but other functions such as root mean square error can also be used. Splits which do not have at least S data points in each subdomain are invalidated by setting their loss to infinity.

The split with the lowest overall error is identified and added to the tree. Each of the two subdomains, if it contains at least $2S$ data points, is added to a queue. The entire procedure, beginning from the identification of quantile splits, is repeated for each subdomain in the queue. As the subdivision process continues, the number of data points in the subdomains will decrease until it is below the $2S$ threshold for all subdomains, at which time the queue will be empty and the recursive algorithm will terminate. Alternatively, a maximum depth can be specified which can serve as a kind of regularization for the tree.

An example of a trained HT is shown in Fig. 4. This example shows various linear combinations of just two input features which are used as decision variables at the tree nodes. There are eight leaves with eight different linear models of the response variable. The figure also demonstrates how the hyperplanes result in a set of disjoint convex polytopes whose union covers the entire domain. A piecewise linear model is obtained by applying the appropriate linear model in each polytope subdomain.

3. Example and tutorial

To demonstrate the usability of the HyperplaneTree package and show the application of the HT surrogates, we present the modeling of an explicit, known function. Eq. (1) shows the function that was used for the example. This function was selected because its nonlinearities arise mainly from interactions between variables, with most terms involving both variables in their formulation. Our investigations indicate that the functions with coupled interactions like these are suited to HT surrogates. They show the most improvement in accuracy for a given number of leaves for an HT versus an LMDT.

$$a = \frac{x - 0.5}{1.5} + y + 0.5, \quad (1a)$$

$$\text{term}_1 = a \cdot x \cdot y, \quad (1b)$$

$$\text{term}_2 = a^2, \quad (1c)$$

$$\text{term}_3 = 3 \cdot \cos(2 \cdot a), \quad (1d)$$

$$\text{term}_4 = 0.2 \cdot (x - 0.5)^2 \cdot (y + 0.5)^2, \quad (1e)$$

$$\text{term}_5 = 0.1 \cdot x^4, \quad (1f)$$

$$\text{term}_6 = 0.1 \cdot y^4, \quad (1g)$$

$$\text{function} = \text{term}_1 + \text{term}_2 + \text{term}_3 + \text{term}_4 + \text{term}_5 + \text{term}_6. \quad (1h)$$

This function has multiple minima in its landscape. This allows us to demonstrate the use of Optimization and Machine Learning Toolkit (OMLT) (Ceccon et al., 2022) to embed the surrogate as the objective function in an optimization problem formulated in the algebraic modeling language Pyomo (Bynum et al., 2021). By employing the piecewise linear surrogates in this simple case, we transform the problem from a nonlinear program (NLP) into a mixed-integer linear program (MILP), which can be readily solved with both commercial and open-source solvers. This is done by first representing the tree as an LMDT with OMLT and converting it to a generalized disjunctive program (GDP) (Chen et al., 2022). In this formulation, there is a set of linear constraints for the hyperplane variables, and each disjunct will have bounds on each of these transformed variables. Then, the GDP is formulated as a set of MILP constraints using transformations such as a convex hull (Lee and Grossmann, 2000) or big-M (Trespalcios and Grossmann, 2015). After this reformulation, each leaf will be assigned a binary variable which serves as an “indicator” of which leaf is “active” (Raman and Grossmann, 1991, 1994).

A Jupyter Notebook with the analyses performed for this example, written in tutorial form and with discussions on hyperparameters sweep can be found in the Supporting Information (SI).

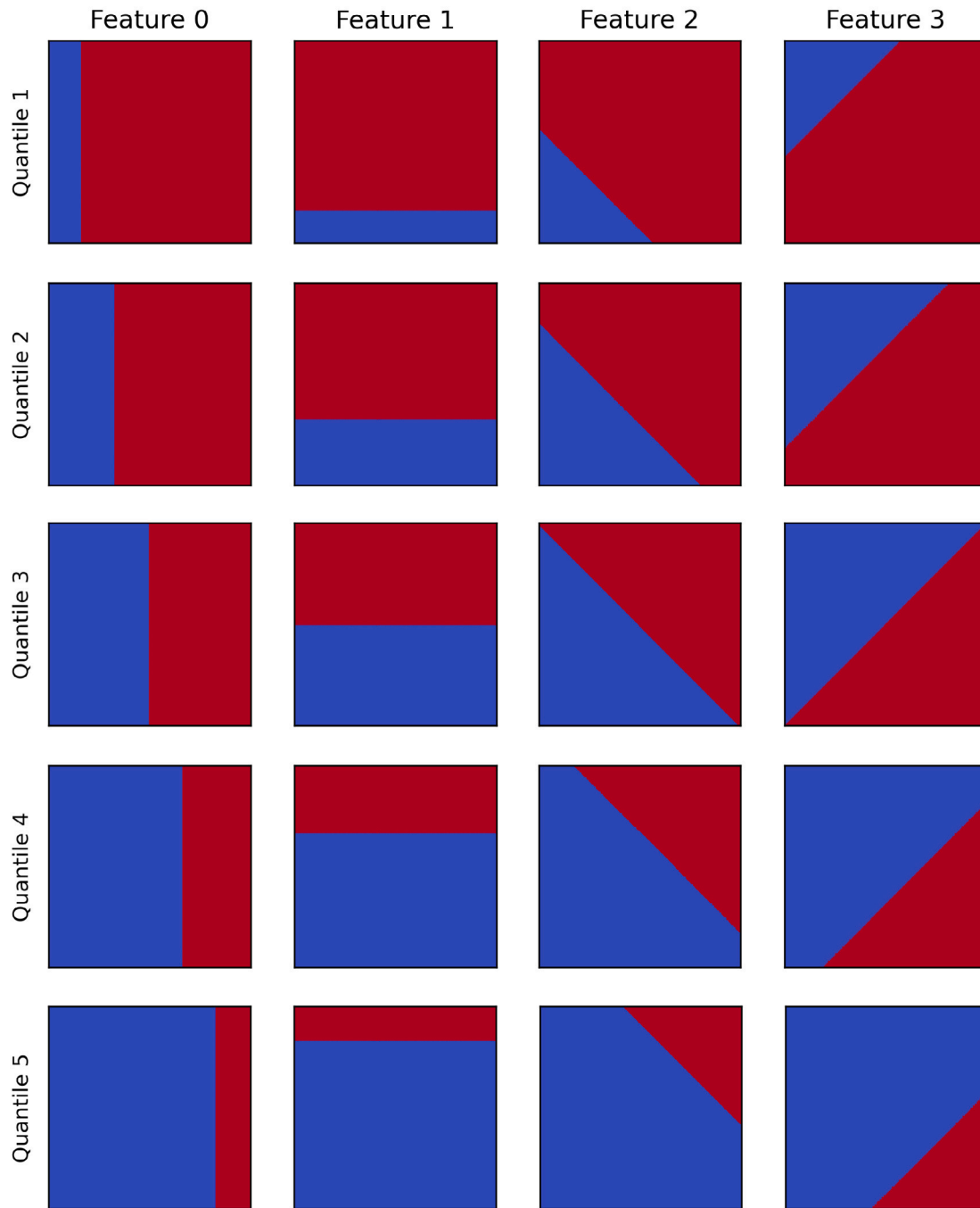


Fig. 3. The set of possible splits is defined by quantiles of each feature. The square represents one subdomain. Each of these possible splits is evaluated and the one with the lowest training loss is selected. Then, this set of possible splits is re-generated and re-evaluated for each of the new subdomains, resulting in a recursive training process. This figure is generated assuming a uniform distribution of data points throughout a square subdomain. In reality, many subdomains will not be rectangular, and the data distribution need not be uniform. Features 0 and 1 are the original features. Features 2 and 3 are the $[1, 1]$ and $[1, -1]$ hyperplanes, respectively. This represents all possible splits for the case: $J = 2$, $W = 1$, $N = 2$, $B = 5$. K , which is a function of J , W , and N , is 4 here.

Fig. 5 shows plots of the original/underlying function alongside surrogates generated using three different modeling approaches: a traditional decision tree, an LMDT, and an HT. The decision tree model will divide the data into subdomains and approximate the response in each subdomain with a constant value. The LMDT makes similar horizontal and vertical splits, but uses a linear model of the response in each subdomain. Finally, the hyperplane tree model enables the use of diagonal, or hyperplane, splits, as demonstrated in Fig. 3. The surrogate's fit to the original function improves as the complexity of the modeling strategy increases. This progression demonstrates how

techniques like LMDT and HT can enable a better description of the underlying function.

Moreover, as the model complexity increases, the surrogate can achieve a better fit with fewer leaves (subdomains). Table 3 highlights how models with similar number of leaves lead to different mean absolute error (MAE) values between responses from the dataset and model predictions, for the particular case of this example. The trade-off is with the training time, which increases with model complexity as more options for splits and detailed models have to be considered. This

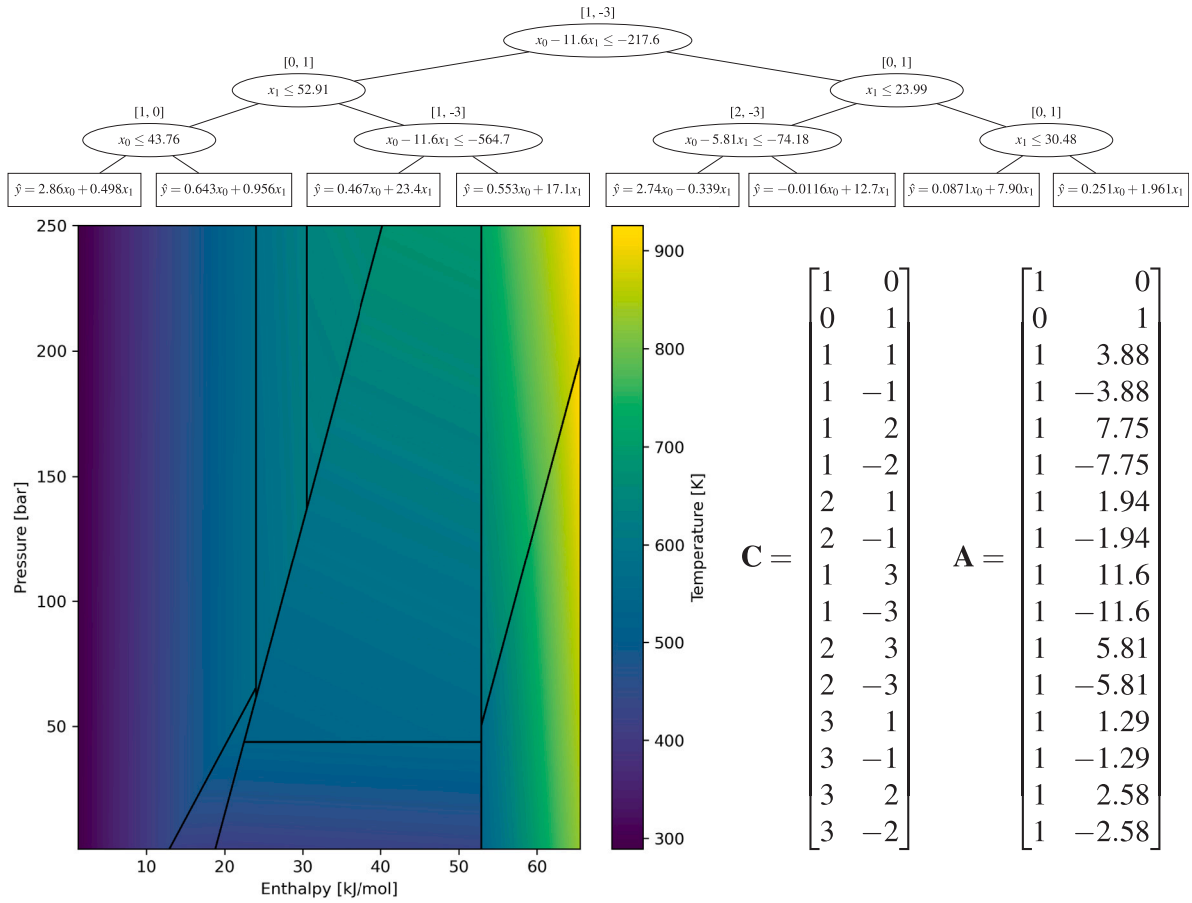


Fig. 4. An example of hyperplane decision tree trained by the procedure outlined in this work. A maximum depth of 3 has been specified. Here, x_0 is enthalpy in kJ/mol, x_1 is pressure in bar, and $\hat{y}$ is a predicted temperature in K. The linear splits are visible in the P-H diagram, and the relevant row of C is annotated above each split in the tree. The weight matrix, C , and normalized linear transformation matrix, A , are shown. This tree is trained on the same dataset described in the Case Study section.

Table 3

Comparison of model performance metrics for the example problem. The hyperparameters have been adjusted to ensure trees have comparable numbers of leaves.

Model	Number of leaves	Error (MAE)	Training time (s)
Decision tree	150	1.180	0.02
Linear tree	149	0.147	0.74
Hyperplane tree	149	0.100	2.30

means more linear regressions have to be performed at each splitting step in the training loop.

Additionally, Fig. 6 illustrates the relationship between tree expansion and MAE for each one of the decision tree-based models. As expected, larger trees generally lead to better fits up until the point in which the test errors begin to increase, indicating model overfitting. The traditional decision tree seems to be the least prone to overfitting in this case, but this is because it is significantly less accurate than the other methods discussed here. The models shown in orange and green are different implementations of LMDTs, so it is expected that their behavior in this analysis is similar. Cerliani's implementation was used in previous work (Ammari et al., 2023; Cerliani, 2024) and served as the inspiration for the implementation in the HyperplaneTree package. HTs, shown in red, are the most accurate for any given number of leaves below about 2000. At this point, the HTs become more prone to overfitting than LMDTs.

The HyperplaneTree package also contains tools that allow the HT surrogates to be embedded in optimization problems via OMLT. Once the piecewise linear model is embedded in the optimization problem, the problem can be solved with MILP solvers such as CBC (Forrest et al.,

2024) and GLPK (FSF, 2012). In this case, using the HT model as a surrogate for the optimization led to an optimal point closer to the true minimum than what is found when using a linear tree surrogate. This is illustrated in Fig. 7, which also depicts the hyperplane subdomains in the final model.

Due to the piecewise (discontinuous) nature of these surrogates, small changes in the input points can lead to significant changes in the response, if this change moves the point into another subdomain. A thorough investigation can be seen in the Jupyter Notebook for this example, which is available in the SI.

This small demonstration shows that these surrogates can be embedded in optimization problems and yield results that are similar to those of LMDTs. Beyond just replacing the objective function in an unconstrained optimization problem, these surrogates can also replace nonlinear, implicit, simulation-based, or data-based constraints in optimization problems. This application is demonstrated and discussed through the case study presented in the following section.

4. Case study: a hyperplane tree surrogate for heat exchanger design

We now present a comprehensive case study to demonstrate the application of a hyperplane tree surrogate in a heat exchanger design problem, leveraging the OMLT package (Ceccon et al., 2022). In this problem, the objective is to minimize the cost of a heat exchanger process within certain specifications and physical constraints. One of these constraints is energy conservation, requiring an estimate of the enthalpy of water. The relationship between pressure, enthalpy, and temperature is nonlinear. Rather than using a nonlinear equation as a

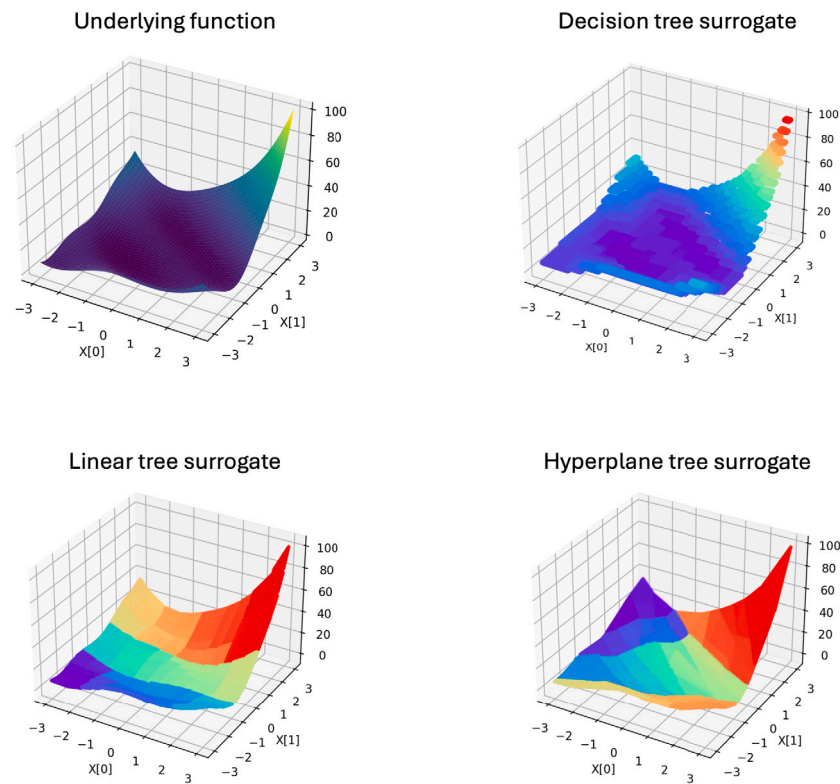


Fig. 5. Tridimensional plots of the underlying function used as working example and the decision tree-based surrogates. For the surrogates, different colors represent individual leaves.

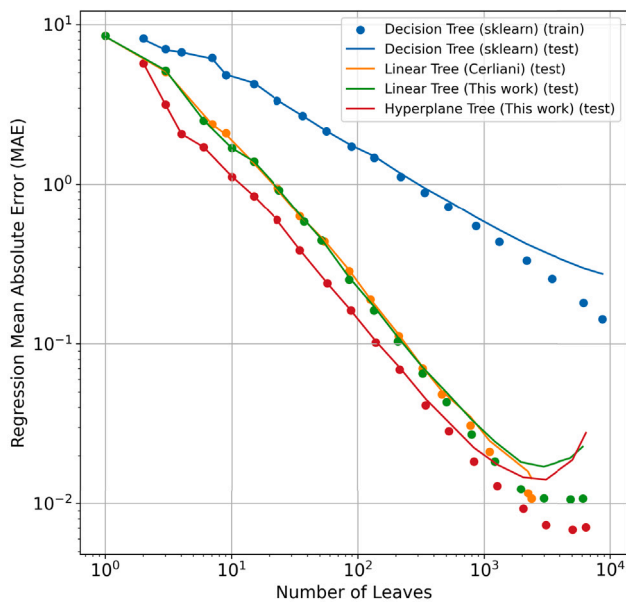


Fig. 6. Analysis of train and test errors for the decision tree-based models for the example case. Dots represent train errors and lines represent test errors. It is possible to identify the maximum number of leaves that can be used before model overfitting.

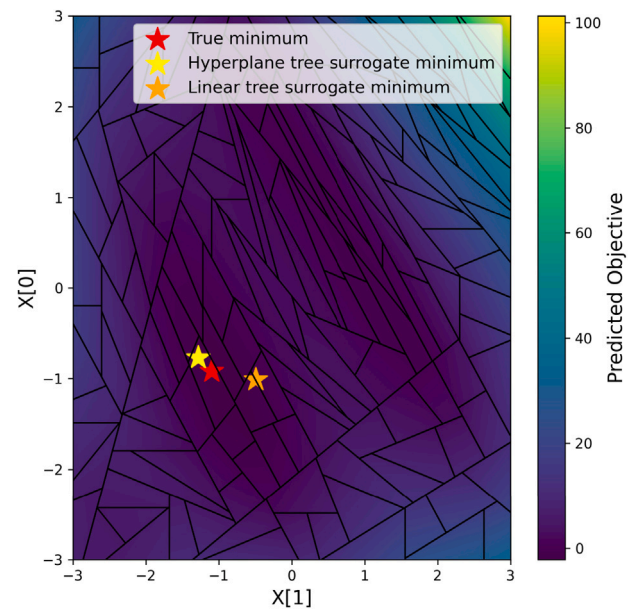


Fig. 7. Visualization of hyperplane subdomains in the response surface. Stars represent the true minimum point of the function (red) and the optimum identified by solving an optimization problem using HT(yellow) and LMDT (orange) surrogates. The true minimum was found using IPOPT, a widely used nonlinear solver (Wächter and Biegler, 2006).

constraint, we implement piecewise linear surrogate models based on tabulated data for these thermodynamic variables. The surrogate model predicts temperature as a function of pressure and enthalpy (PHT) for water. The regression dataset contains 10,000 rows and 3 columns: pressure in Pascal, enthalpy in Joules per mole, and temperature in Kelvin. It is sampled on an evenly spaced 100 by 100 grid of pressures

ranging from 100,000 Pa to 25,000,000 Pa and enthalpies ranging from 1275 J/mol to 65 523 J/mol. At each of these points a temperature is computed from the International Association of the Properties of Water and Steam (IAPWS-97) model (Wagner et al., 2000). This heat

Table 4

Comparison of model performance metrics for the PHT surrogate. As the model complexity increases, both number of leaves and MAE values decrease and training times increase.

Model	Number of leaves	Error (MAE)	Training time (s)
Decision tree	469	1.961	0.01
Linear tree	181	0.521	0.47
Hyperplane tree	119	0.453	1.48

Table 5

Comparison of model performance metrics for the PHT surrogate. The hyperparameters have been adjusted to ensure trees have comparable numbers of leaves.

Model	Number of leaves	Error (MAE)	Training time (s)
Decision tree	120	5.488	0.00
Linear tree	119	0.776	0.38
Hyperplane tree	119	0.453	1.48

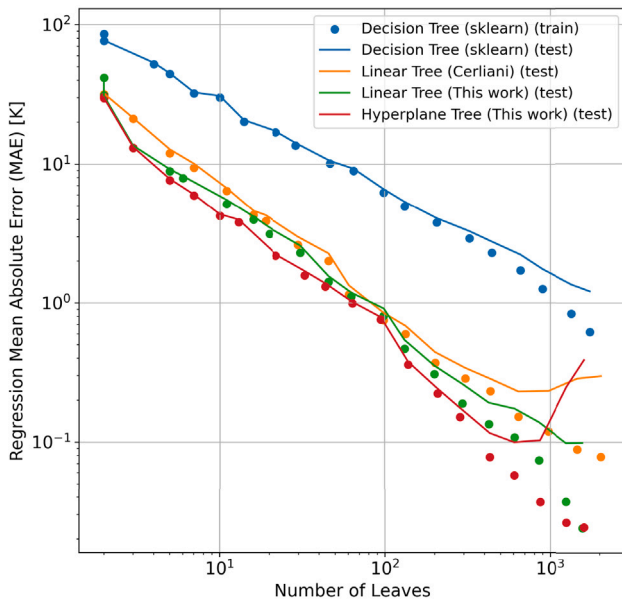


Fig. 8. Analysis of train and test errors for the decision tree-based models for the PHT case study. Dots represent train errors and lines represent test errors. It is possible to identify the maximum number of leaves that can be used before model overfitting.

exchanger design problem was originally implemented in the IDAES framework (Lee et al., 2021). Further details on the scope and formulation of the engineering optimization problem are available in prior work, which explored the use of LMDTs in the same problem (Ammari et al., 2023).

The goal of this case study is to evaluate and compare the performance and outcomes of the surrogate models and their application in a global optimization framework. Table 4 presents the results for the three different regression models. Among these, the hyperplane tree achieves the lowest mean absolute error while also requiring the fewest leaves. As for the previous example case, the trade-off is given by the increased time to train the model. The added complexity of incorporating the linear combinations in the surrogate extends the time needed to select a linear model for each leaf.

To generate the results presented in Table 5, we managed the hyperparameters of the decision and linear trees to ensure a comparison between trees of similar sizes. Fig. 8 illustrates the trade-off analysis between MAE and number of leaves. The observed trend aligns with the example discussed in the previous section, where the HT surrogate generally outperforms the linear tree models, and overfitting is observed beyond certain tree sizes.

Table 6

Comparison of models' performance when embedded in the heat exchanger design problem and optimal results. Hyperplane trees with a maximum hyperplane weight (W) of 2 and 3 are compared. This has a significant impact on the runtime of the optimization without much effect on the optimal point found.

Surrogate	LMDT	HT $W = 3$	HT $W = 2$	ANN	No surrogate
Total cost (millions of \$)	4.76	4.76	4.76	4.75	4.78
Steam flow (kg/s)	60.08	60.09	60.09	60.02	60.31
Heat exchanger area (m ²)	2915	2919	2921	2888	2924
Number of leaves	337	168	174	–	–
MAE (K)	0.30	0.30	0.28	1.92	–
Training time (s)	3.6	7.1	2.9	56.4	–
BARON memory usage (MB)	6	11	8	7	–
BARON run time (s)	1.3	6.4	1.4	5097.9	–

Fig. 9 shows the phase diagrams obtained from training a linear tree surrogate (comparable to the Figure 6 of prior work (Ammari et al., 2023)), and hyperplane trees with different number of leaves. The black lines indicate the leaf divisions and splits. Across the three plots, the rectangular and horizontal behavior of the leaves in the vapor-liquid equilibrium (VLE) zone is clearly visible. The HT surrogates appear to be particularly valuable in capturing the behavior on the boundary of the VLE envelope, where we see large diagonal lines in the HT case and many small subdomains in the LMDT case.

Once the PHT surrogates are trained, they are embedded in the heat exchanger design problem using OMLT. In the original formulation, the PHT relationship was represented as a simulation-based constraint, and the formulation as a whole made the problem an NLP. By introducing the piecewise linear model derived from the tree, the problem becomes a mixed-integer nonlinear program (MINLP), which requires the use of global solvers, such as BARON (Tawarmalani and Sahinidis, 2005) or the open-source solver SCIP (Achterberg, 2009).

Table 6 shows the results for the original NLP formulation, meaning there is no surrogate for the PHT relation, as mentioned in previous work (Ammari et al., 2023), alongside results obtained using LMDT, HT surrogates and an artificial neural network (ANN). The optimal results obtained with the surrogates are within 1% the original NLP. We show two different HT surrogates, with different values for the maximum hyperplane weight (W). The HT surrogates have significant fewer leaves than the LMDT surrogate, while still having similar MAE values.

Despite their smaller size, with the reformulation used, the HTs are similarly computationally expensive. This may be due to the increased complexity of the constraints for the hyperplane splits or the increased number of variables due to the linear transformations to capture the hyperplane splits. In this heat exchanger design case study, the surrogate has two input variables. With the default value of $W = 3$, the hyperplanes introduce 14 additional variables. These 14 variables come along with 14 new sets of bounds within each leaf, and 14 new linear constraints which are applied outside of the tree formulation. In the case where $W = 2$, only six new variables are introduced and the solve time is competitive with the linear-tree case. Future work is necessary to see if other reformulations of the HT result in more competitive performance.

These results contrast with those obtained using the ANN with rectified linear unit (ReLU) activation as a surrogate. To achieve a comparable MAE to the tree surrogates, we trained a surrogate with four layers, and 200, 100, 50, and 25 neurons, respectively (375 total nonlinearities). These and other hyperparameters were selected through hyperparameter grid search. The end depth of the surrogate leads to the higher training and optimization run times. We emphasize that using different hyperparameters for training can lead to different training and optimization run times, and different results. The comparison further strengthens the advantages of using the tree-based surrogates over ANNs for this case.

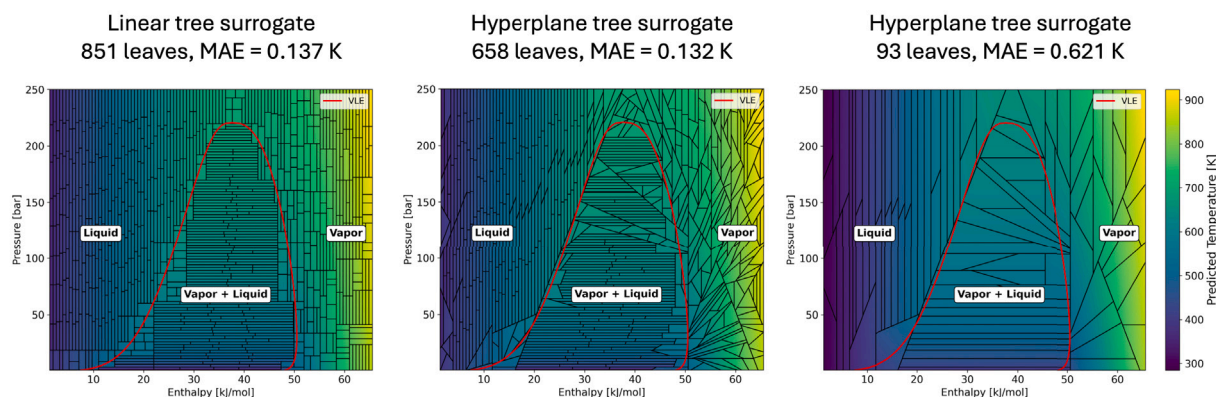


Fig. 9. Phase diagrams with the rectangular and hyperplane subdomains in the response surface for the PHT surrogate. Indications for the number of leaves on the decision trees and MAE provide intuition into the efficiency of the models in fitting the data. The red line marks the division between liquid and vapor phases and the vapor–liquid equilibrium, as given by IAPWS97 tables (Wagner et al., 2000).

Through this case study, we demonstrated the advantages of employing the more sophisticated fitting strategy of HTs. The Jupyter Notebooks for this case study, available in the SI, provides a detailed walk-through of the training, embedding, and optimization procedures, along with discussions and the generation of all figures presented in this part of the work.

5. Conclusions

In this work, we introduced the algorithm and methodology for building hyperplane decision tree models, implemented in the HyperplaneTree package. This package is available at <https://github.com/LLNL/systems2atoms/tree/main/systems2atoms/hyperplanetree>. Like linear model decision trees, these models approximate responses at the leaves. The key modeling improvement is to introduce additional features which are just linear combinations of the input variables. This adds new ways that the tree could split the data, often leading to better accuracy for a given number of leaves, while retaining the ability of the surrogate to be formulated as a mixed-integer linear constraint.

We investigated an example and a case study comparing the performance of hyperplane trees with traditional decision trees and linear model decision trees. The results generally demonstrate that the hyperplane tree modeling leads to smaller trees and lower MAE values, with the trade-off of increased training time. However, we reiterate that the improvements and advantages regarding surrogate accuracy or model size when using hyperplane tree models over linear tree models are likely case dependent.

The primary application of these surrogate modeling strategies, both LMDT and HT, is to replace nonlinear, simulation-based, or otherwise difficult-to-formulate expressions in optimization problems. The use of these surrogates may improve solution time, or even enable solutions of problems that might otherwise be computationally intractable. The suitability of linear or hyperplane tree models for a specific optimization problem can depend on many factors. For instance, for a specific case, hyperplane tree models may be more accurate and/or smaller, but their increased complexity can result in slower optimization solves.

The Jupyter Notebooks provided in the SI are useful for demonstrating in a straightforward manner how to train the decision tree-based models and integrate them into optimization problems using OMLT. We encourage users to explore both modeling techniques and hyperparameters, and evaluate their suitability for their specific application.

CRediT authorship contribution statement

Ethan M. Sunshine: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Carolina Colombo**

Tedesco: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Sneha A. Akhade:** Writing – review & editing, Supervision, Resources, Project administration, Investigation, Funding acquisition. **Matthew J. McNelly:** Writing – review & editing, Supervision, Resources, Project administration, Investigation, Funding acquisition. **John R. Kitchen:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization. **Carl D. Laird:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344 and was supported by Laboratory Directed Research and Development (LDRD) funding under project 23-ERD-016. This work has been assigned an LLNL Information Management release number of LLNL-JRNL-2001796. The authors thank Bashar Ammari for helpful discussions and for providing code files used in previous work (Ammari et al., 2023).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.compchemeng.2025.109204>.

Data availability

Link provided to the repository in the publication.

References

- Achterberg, Tobias, 2009. SCIP: solving constraint integer programs. *Math. Program. Comput.* 1 (1), 1–41.
- Ammari, Bashar L., Johnson, Emma S., Stinchfield, Georgia, Kim, Taehun, Bynum, Michael, Hart, William E., Pulsipher, Joshua, Laird, Carl D., 2023. Linear model decision trees as surrogates in optimization of engineering applications. *Comput. Chem. Eng.* 178, 108347.
- Ashcroft, Neil, Mermin, N. David, 1976. *Solid State Physics*. Holt, Rinehart, and Winston, New York, NY.

- Bertsimas, Dimitris, Dunn, Jack, Wang, Yuchen, 2021. Near-optimal nonlinear regression trees. *Oper. Res. Lett.* 49 (2), 201–206.
- Bollwein, Ferdinand, 2024. A pivot-based simulated annealing algorithm to determine oblique splits for decision tree induction. *Comput. Statist.* 39 (2), 803–834, Publisher: Springer Science and Business Media LLC.
- Bynum, Michael L., Hackebeil, Gabriel A., Hart, William E., Laird, Carl D., Nicholson, Bethany L., Sirola, John D., Watson, Jean-Paul, Woodruff, David L., 2021. *Pyomo — Optimization modeling in python*. In: Springer Optimization and Its Applications, vol. 67, Springer International Publishing, Cham.
- Ceccon, Francesco, Jalving, Jordan, Haddad, Joshua, Thebelt, Alexander, Tsay, Calvin, Laird, Carl D, Misener, Ruth, 2022. OMLT: optimization & machine learning toolkit. *J. Mach. Learn. Res.* 23 (1), 349:15829–349:15836.
- Cerliani, Marco, 2024. Cerlymarco/linear-tree. original-date: 2021-03-27T08:57:48Z.
- Chen, Qi, Johnson, Emma S., Bernal, David E., Valentin, Romeo, Kale, Sunjeev, Bates, Johnny, Sirola, John D., Grossmann, Ignacio E., 2022. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. *Optim. Eng.* 23 (1), 607–642.
- Cozad, Alison, Sahinidis, Nikolaos V., Miller, David C., 2014. Learning surrogate models for simulation-based optimization. *AIChE J.* 60 (6), 2211–2227.
- Forrest, John, Ralphs, Ted, Vigerske, Stefan, Santos, Haroldo Gambini, Forrest, John, Hafer, Lou, Kristjansson, Bjarni, jpfasano, EdwinStraver, Jan-Willem, Lubin, Miles, rlougee, a andre, jgponcall, Brito, Samuel, h-i gassmann, Cristina, Saltzman, Matthew, tosttost, Pitrus, Bruno, Matsushima, Fumiaki, Vossler, Patrick, SWGY, Ron @, to st, 2024. coin-or/Cbc: Release releases/2.10.12.
- FSF, 2012. GLPK - GNU linear programming kit.
- Ganaie, M.A., Tanveer, M., Suganthan, P.N., 2020. Oblique decision tree ensemble via twin bounded SVM. *Expert Syst. Appl.* 143 (113072), 113072, Publisher: Elsevier BV.
- Georgiou, Angelos, Wiesemann, Wolfram, Kuhn, Daniel, 2015. Generalized decision rule approximations for stochastic programming via liftings. *Math. Program.* 152 (1–2), 301–338.
- Grimstad, Bjarne, Andersson, Henrik, 2019. ReLU networks as surrogate models in mixed-integer linear programs. *Comput. Chem. Eng.* 131, 106580.
- Han, Eojin, Nohadani, Omid, 2024. Nonlinear decision rules made scalable by nonparametric liftings. *Manag. Sci.* Publisher: INFORMS.
- Juez-Gil, Mario, Arnaiz-González, Álar, Rodríguez, Juan J., López-Nozal, Carlos, García-Osorio, César, 2021. Rotation forest for big data. *Inf. Fusion* 74, 39–49.
- Lee, Andrew, Ghouse, Jaffer H., Eslick, John C., Laird, Carl D., Sirola, John D., Zamarripa, Miguel A., Gunter, Dan, Shinn, John H., Dowling, Alexander W., Bhattacharyya, Debansu, Biegler, Lorenz T., Burgard, Anthony P., Miller, David C., 2021. The IDAES process modeling framework and model library—Flexibility for process simulation and optimization. *J. Adv. Manuf. Process.* 3 (3), e10095.
- Lee, Sangbum, Grossmann, Ignacio E., 2000. New algorithms for nonlinear generalized disjunctive programming. *Comput. Chem. Eng.* 24 (9), 2125–2141.
- Lee, Guang-He, Jaakkola, Tommi S., 2020. Oblique decision trees from derivatives of ReLU networks.
- Pedrozo, H.A., Rodriguez Reartes, S.B., Chen, Q., Diaz, M.S., Grossmann, I.E., 2020. Surrogate-model based MILP for the optimal design of ethylene production from shale gas. *Comput. Chem. Eng.* 141, 107015.
- Rahal, Said, Li, Zukui, Papageorgiou, Dimitri J., 2022. Deep lifted decision rules for two-stage adaptive optimization problems. *Comput. Chem. Eng.* 159, 107661.
- Raman, R., Grossmann, I.E., 1991. Relation between MILP modelling and logical inference for chemical process synthesis. *Comput. Chem. Eng.* 15 (2), 73–84.
- Raman, R., Grossmann, I.E., 1994. Modelling and computational techniques for logic based integer programming. *Comput. Chem. Eng.* 18 (7), 563–578.
- Raymaekers, Jakob, Rousseeuw, Peter J., Verdonck, Tim, Yao, Ruicong, 2024. Fast linear model trees by PILOT. *Mach. Learn.* 113 (9), 6561–6610.
- Rodriguez, J.J., Kuncheva, L.I., Alonso, C.J., 2006. Rotation forest: A new classifier ensemble method. *IEEE Trans. Pattern Anal. Mach. Intell.* 28 (10), 1619–1630, Conference Name: IEEE Transactions on Pattern Analysis and Machine Intelligence.
- Setiono, R., Liu, H., 1999. A connectionist approach to generating oblique decision trees. *IEEE Trans. Syst. Man Cybern. B Cybern.* 29 (3), 440–444, Publisher: Institute of Electrical and Electronics Engineers (IEEE).
- Shah, S., Sastry, P.S., 1999. New algorithms for learning and pruning oblique decision trees. *IEEE Trans. Syst. Man Cybern. Part C (Appl. Rev.)* 29 (4), 494–505, Conference Name: IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews).
- Tawarmalani, Mohit, Sahinidis, Nikolaos V., 2005. A polyhedral branch-and-cut approach to global optimization. *Math. Program.* 103 (2), 225–249.
- Teichrib, Dieter, Darup, Moritz Schulze, 2024. Piecewise regression via mixed-integer programming for MPC. In: *Proceedings of the 6th Annual Learning for Dynamics & Control Conference*. PMLR, pp. 337–348, ISSN: 2640-3498.
- Trespalcacios, Francisco, Grossmann, Ignacio E., 2015. Improved Big-M reformulation for generalized disjunctive programs. *Comput. Chem. Eng.* 76, 98–103.
- Wächter, Andreas, Biegler, Lorenz T., 2006. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Math. Program.* 106 (1), 25–57.
- Wagner, W., Cooper, J.R., Dittmann, A., Kijima, J., Kretschmar, H.-J., Kruse, A., Mares, R., Oguchi, K., Sato, H., Stocker, I., Sifner, O., Takaishi, Y., Tanishita, I., Trübenbach, J., Willkommen, Th., 2000. The IAPWS industrial formulation 1997 for the thermodynamic properties of water and steam. *J. Eng. Gas Turbines Power* 122 (1), 150–184.
- Wickramarachchi, D.C., Robertson, B.L., Reale, M., Price, C.J., Brown, J., 2015. HHCART: An oblique decision tree.